

결과와 분위기 모두 만들고 싶은 개발자입니다.

이름: 김기태

연락처: 010.2061.7022

이메일: kku98057@naver.com

소개

스타트업에서의 다양한 협업 경험과 직접 서비스를 기획·개발·운영한 경험을 통해 **프로젝트에 대한 높은 오퍼십과 주도적인 문제 해결 능력**을 갖추게 되었습니다.

단순한 기능 구현을 넘어, **사용자 경험(UX), 데이터 흐름, 수익 구조 등 비즈니스적인 관점에서 기능의 효과를 고민하며 개발하는 것**을 중요하게 생각합니다.

항상 **사용자의 입장에서 사고하고, 개발이 서비스 성과에 어떤 영향을 주는지 고민하며 일합니다.**

퍼블리셔 **역할을 수행하며** 자연스럽게 React와 TypeScript 기반의 프론트엔드 개발까지 함께 경험해왔다보니 퍼블리싱에도 **강점**이 있습니다.

앱 운영 경험

직접 기획 및 개발한 앱을 출시하여 **약 5천 다운로드, 평점 4.7점**을 기록하며 운영해본 경험이 있습니다.

배포부터 업데이트, 사용자 CS 대응까지 전 과정을 직접 경험하였으며, 사용자 피드백을 빠르게 반영하고 예상치 못한 이슈에 유연하게 대응하면서 **실전 문제 해결 능력과 제품 개선 감각**을 키웠습니다

기술 스택

- | | | |
|---------------|-----------|--------------|
| • HTML | • CSS | • Javascript |
| • Typescript | • React | • Nextjs |
| • react-query | • Threejs | • zustand |

- Recoil
- tailwind

경력

팀명 : 팀 줍줍

2025.01.15 ~ 현재 | 프론트엔드

미션형 앱 테크 신규 개발

개발환경 : React, vite, typescript

라이브러리 : React-query, zustand, tailwind, chartjs, axios, lottie, framer-motion

주요업무

- 프로젝트 구조 설계 및 컴포넌트 개발
- 구글, 카카오, 이메일 회원가입 및 로그인 구현
- Flutter 브릿지를 통한 데이터 통신
- 어드민 개발
- 미션 및 클릭형 게임 개발
- 구글 광고 연동

트러블슈팅

▼ 업데이트 버전 불일치 문제

원인분석

- SPA의 정적 자원이 캐시되어 있어, 페이지 URL이 변경되더라도 최신 콘텐츠가 반영되지 않는 현상 발생
- 사용자가 새롭게 앱을 실행 하지 않는 이상 업데이트가 적용되지 않아, 버전 불일치로 인한 UX 저하가 지속됨

해결

- 앱이 일정 주기마다, 또는 백그라운드에서 포그라운드로 전환될 때 `version.json` 을 호출하여 최신 버전을 확인

- 버전 정보가 불일치할 경우, 자동으로 페이지를 리로드하거나 안내를 통해 유저가 새로고침을 하도록 유도
- 해당 로직은 **클라이언트에서만 동작**하므로 서버 CPU/메모리 리소스에는 영향을 주지 않으며, 네트워크 트래픽 또한 매우 경량으로 효율적인 구조

성과

- 업데이트 시 버전 불일치 상황 제거로 인한 CS문의 100% 제거

▼ 초기 로딩속도 개선 및 메모리 캐시구현

초기 페이지 로딩 속도가 매우 느려 사용자 경험에 영향을 주는 이슈가 있었고 앱 내 다수의 Lottie 애니메이션 파일 사용으로 인한 **초기 번들 크기 증가**와 동일한 애니메이션을 여러 컴포넌트에서 재사용할 때마다 **로딩 상태 표시 → 애니메이션 렌더링** 과정에서 깜빡임 현상이 발생하여 사용자 경험을 저해하였습니다.

원인분석

- 진단 결과 `index.js` 번들 파일 용량이 과도하게 커져 있는 것을 확인
- 라이브러리 단위로 청크 분리(chunk splitting)를 시도했지만 **효과가 미미**
- `src` 폴더 내부에 포함된 **1MB 이상의 Lottie JSON 파일**이 번들 파일 내에 포함되고 있는 것을 발견
- 동일한 Lottie애니메이션을 사용하는 각 컴포넌트마다 독립적인 상태 관리로 인한 **중복 로딩 현상 발생**

해결

- 해당 Lottie파일을 **public 폴더로 이동**해 번들링에서 제외.
- **모듈 레벨 캐싱 시스템**을 설계 및 구현하여, 동일한 Lottie 애니메이션에 대해 **메모리 캐시**를 우선 조회하도록 처리

성과

- 초기 로딩 시간 **약 120% 단축**
- 이미 로드된 애니메이션의 **깜빡임 현상 100% 제거** 및 즉시 렌더링 구현으로 **사용자 경험 개선**
- **메모리 효율 향상**

▼ 강화게임 보상설계

강화게임 내 보상 설계는 **수익성과 게임 밸런스**의 핵심 요소로, 부적절한 설계 시 **손실이 발생할 위험**이 있었습니다. 특히 광고 수익은 **동일 유저**에 대해 **퀄리티가 낮게 평가**되어 매우 낮은 수준으로 지급되기 때문에, 신규 유저 유입 감소에 따른 **장기적인 수익 감소 문제**가 발생할 수 있었습니다. 이에 따라 근본적인 밸런스 조정을 위해 **3단계**에 걸쳐 보상 설계를 진행하였습니다.

원인분석

- 동일한 유저가 반복적으로 광고를 시청할 경우 eCPM(1,000회 노출당 광고 수익)이 **지속해서 하락**하는 문제 발생
- 시간이 지날수록 신규 유저 유입이 줄어들면서 전체 수익성 저하 우려

해결

- 1차: 임의의 보상 설계 초안 생성
- 2차: 파이썬을 활용해 유저 평균 수익을 목표 수익 기준에 맞게 산출 및 조정
- 3차: 직접적인 테스트와 데이터 분석을 통한 추가 밸런스 조정

성과

- 유저가 획득하는 수익 대비 **약 3배 수준의 수익 전환**을 달성

▼ 전반적인 광고 수익의 하락화

광고 수익은 동일한 유저가 반복 노출될 경우 광고 플랫폼에서 낮은 eCPM이 부여되어 전체 수익이 감소하는 구조적 문제를 가지고 있습니다. 이를 극복하기 위해 여러 광고 플랫폼을 연동하는 **미디어이션 전략**을 도입하였고, **네트워크 다양화**를 통해 보다 **높은 입찰가의 광고를 유저에게 제공**함으로써 광고 수익 극대화를 도모하였습니다.

원인분석

- 동일한 유저가 반복적으로 광고를 시청할 경우 eCPM이 점진적으로 하락
- 신규 유저 유입 감소 시 광고 수익 전체 하락 위험

해결

- 타 광고 플랫폼과 현재 사용 중인 광고 플랫폼을 미디어이션을 통해 eCPM 상승 유도
- 네트워크 다양화로 높은 입찰가 광고 노출을 확대

성과

- 수익 **약 1.5배에서 2배 가량 증가** 효과 달성

회사명: 프리윙

2024.5.15 ~ 2024.11.30 | 프론트엔드

영상 데이터 콘텐츠 기반 국민 여론 플랫폼 신규 웹 개발

개발환경 : Nextjs, React, vite, typescript

라이브러리 : React-query, Recoil, tailwind, chartjs, axios, lottie

주요업무

- 프로젝트 구조 설계 및 컴포넌트 개발
- 구글, 카카오, 이메일 회원가입 및 로그인 구현
- 결제 연동
- 상점 기능 개발
- 투표, 게시판, 댓글 기능 개발
- 어드민 개발

트러블슈팅

▼ 초기 로딩 속도 개선

배포 이후 초기 로딩 속도가 비정상적으로 3초나 소요되는 문제가 발생하여, 사용자 경험 저하와 서비스 품질 하락 우려가 있었습니다. 초기 대응으로 라이브러리별 코드 스플릿을 시도했으나 큰 효과를 보지 못해 근본 원인 분석에 집중하였습니다.

원인분석

- 초기 로딩 속도가 과도하게 느려 3초나 소요되는 비정상적인 현상 발생
- 코드 스플릿 적용에도 불구하고 로딩 속도 개선이 미미함
- CI/CD 환경에서 빌드된 산출물(dist 폴더 내 파일)을 실행해야 하나, 실제로는 빌드되지 않은 파일을 `npm run dev` 로 실행하고 있었음

해결

- 문제의 근본 원인을 파악하여 CI/CD 파이프라인에서 올바른 빌드 산출물이 실행되도록 수정

- 수정 이후 초기 로딩 속도가 3초에서 0.3초로 대폭 개선되어 빠른 서비스 접속 환경을 구현

▼ React Query의 캐싱 사용 개선

React Query를 활용하여 데이터 **fetch시 캐싱**을 적절히 사용함으로써 사용자 경험을 개선하고 서버 부하를 줄이는 방안을 고민하였습니다. 특히, 화면 업데이트가 느린 상세 페이지나 공지사항 등에는 캐싱을 적용하여 빠른 화면 전환을 유도하였으며, 댓글과 게시글에서도 캐싱을 적용했으나 **유저 행동에 따른 캐시 초기화 시점에 문제**가 있었습니다.

원인분석

- 댓글 작성 시에 캐시를 무효화(invalidate)하도록 설정되어, 댓글 작성 경험에 없는 일반 유저는 항상 캐싱된 구버전 데이터를 보게 됨
- 이로 인해 실시간 **최신 데이터 반영이 어렵고**, 사용자 경험 저하 문제 발생
- 반대로 캐싱을 완전히 해제하면, 댓글 화면에서 매번 API 호출과 로딩 화면이 나타나 사용자 불편과 서버 부하 증가 문제가 발생

해결

- 댓글과 게시글 캐싱을 해제하여 실시간 데이터 반영을 우선시하였으나, **사용자의 불편과 서버 리소스 낭비 문제**를 인지
- 이에 **새로고침 버튼을 별도로 구현**하여, 사용자가 직접 캐시를 초기화하고 최신 데이터를 불러올 수 있도록 개선
- 이를 통해 기본적으로는 빠른 데이터 조회가 가능하며, 사용자가 필요 시 직접 최신 데이터를 확인할 수 있게 하여 UX와 성능 간 적절한 균형을 맞춤

고민했던것들

▼ 컴포넌트 설계에 대한 전략적 접근

초기에는 화면 기준이 아닌 **기능 단위(Feature-Based)**로 컴포넌트를 구성하고, 반복되거나 독립적인 비즈니스 로직은 **Custom Hook**으로 분리하는 방식을 주로 사용하였습니다. 이 방식은 유지보수성과 재사용성을 높이는 데 도움이 되었으며, 컴포넌트의 책임을 명확히 구분하는 구조적 이점이 있었습니다.

고민

- 그러나 프로젝트가 진행되며 예상치 못한 기획 변경, UI 개편, 기능 추가가 잦아졌고, 만들어둔 컴포넌트들이 실제로는 거의 사용되지 않거나 전면 수정이 필요해지는 일이 반복되었습니다. 사전에 많은 컴포넌트를 만들어두는 방식이 오히려 생산성을 저하시킨다는 점을 느꼈습니다.

선택 및 결과

이에 따라 무조건적인 컴포넌트화는 지양하고, 프로젝트 흐름을 따라가며 **실제 반복되는 UI나 기능이 보일 때에만 컴포넌트를 분리**하는 방식으로 전략을 전환하였습니다. 이 접근은 빠른 기획 변경에 유연하게 대처할 수 있었고, 불필요한 코드 분리 없이 오히려 더 빠른 개발 속도와 유지보수의 효율성을 확보할 수 있었습니다.

▼ 폴더 구조 및 컴포넌트 관리 전략

리액트 프로젝트를 진행하며 디렉토리 구조 설계에 대한 고민이 있었습니다. 많은 프로젝트에서 공통적으로 사용하는 구조는 루트 디렉토리에 `pages`, `components`, `hooks`, `lib` 등을 위치시키고, `components` 폴더 안에 모든 컴포넌트를 모아 관리하는 방식이었습니다. 이는 프로젝트 초반에는 명확해 보이지만 규모가 커질수록 복잡도가 증가하고, 컴포넌트의 실제 사용처를 파악하기 어려운 단점이 있었습니다.

고민

예를 들어 A 페이지 및 그 하위 페이지에서만 사용하는 컴포넌트까지도 전역 `components` 폴더에 위치시키는 것이 과연 적절한가에 대한 의문이 들었습니다. 이렇게 되면 컴포넌트의 사용 범위가 모호해지고, 유지보수 시 수정이 다른 페이지에 영향을 줄 수 있는지 판단하기 어려워졌습니다.

선택 및 결과

이에 따라 저는 Next.js의 구조에서 영감을 받아 다음과 같은 방식을 채택하였습니다

- `pages` 폴더에서 실제 페이지 단위를 구성
- 특정 페이지에서만 사용하는 컴포넌트는 해당 페이지 폴더 내에 `components` 하위 폴더를 만들어 함께 관리
- 공통적으로 사용되는 컴포넌트만 `src/components` 와 같은 전역 폴더로 이동

이 구조를 적용한 이후, 각 페이지에서 사용하는 컴포넌트를 한눈에 파악하기 쉬워졌고, 유지보수나 리팩토링 시 변경의 영향 범위를 빠르게 추적할 수 있어 개발 생산성이 향상되었습니다.

회사명: (주)선후

2022.2.14 ~ 2024.4.30 | 퍼블리셔 | 프론트엔드

결혼장려 앱 어드민 개발

개발환경 : React, vite, typescript

라이브러리 : React-query, Recoil, axios, chartjs

주요업무

- CRUD위주의 개발
 - 회원관리, 매칭관리 등

트러블슈팅

▼ 페이지네이션 뒤로가기 이슈

각 게시판에서 다음 페이지를 이동 후 뒤로가기를 하는 경우 이전 번호의 페이지가 나와야하지만 이전 게시판페이지로 이동되는 이슈

원인분석

- **URL 변경 없이** 상태만으로 페이지 번호를 변경하고 API를 호출하는 방식으로 구현되어 있었음
- 이로 인해 **브라우저의 히스토리 스택에 페이지 번호가 기록되지 않아**, 뒤로가기시 현재 게시판 페이지의 이전 상태로 돌아가는 것이 아닌, 이전 라우트(게시판)로 돌아가버리는 문제가 발생.

해결

- 페이지 번호를 클릭하면 `useSearchParams()` 의 `setSearchParams` 를 통해 **query parameter** (예: `?page=2`)가 변경되도록 수정.
- query parameter를 react-query의 **queryKey에 포함시켜** page 전환 시 데이터가 업데이트 되도록 처리

Akasic랜딩페이지

개발환경 : **vanillaJS**

라이브러리 : **threejs, gsap, rocomotive-scroll**

주요업무

- 5가지 3D모델을 파티클화처리
- gsap 및 glsl를 활용한 스크롤이벤트로 파티클화 된 모델을 자연스럽게 변형되도록 구현
- 후처리(postprocessing) 기능으로 빛나는 효과 추가

트러블슈팅

▼ 인터렉션 효과 찾기

threejs를 이용하여 레퍼런스를 받은 웹사이트의 인터렉션을 구현함에 있어 이 효과를 어떻게 검색하고 찾아야할지 조차 알 수 없었습니다. 그래서 해당 웹사이트의 소스를 분석하였고 번들링으로 인해 압축이 되어있었지만 운이 좋게도 **morph** 라는 키워드를 발견하여 그것을 위주로 검색하기 시작하였습니다.

원인분석

- 인터렉션을 구현함에 있어 명확히 어떤 명칭으로 검색해야할지 알 수 없고 생각나는 모든 키워드를 입력해봤지만 검색결과가 나오지않음.

해결

- 번들링 된 파일을 뜯어 **morph** 라는 키워드를 얻어 관련된 모든 자료를 검색하여 구현

▼ 모델의 파티클화

threejs에서 지원하는 **PointMaterial** 이라는 생성자 함수가 있지만 이것으로는 해당 효과를 구현하기는 불가능하였습니다. 파티클화된 모델이 다른 모델로 변형될때 x,y,z값이 반드시 0을 향하는 문제가 있어 자유롭게 원하는 변형효과를 만들 수 없었습니다. 따라서 **ShaderMaterial** 생성자 함수를 사용하고 GLSL기반으로 보다 자연스러운 효과를 만들 수 있었습니다.

원인분석

- threejs에서 지원하는 **PointMaterial** 로는 원하는 효과를 기대할 수 없음

해결

- ShaderMaterial 생성자 함수와 GLSL이라는 shading language를 통해 효과 개발